中山大學
SUN YAT-SEN UNIVERSITY

Lecture 18
Multiway Trees



Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

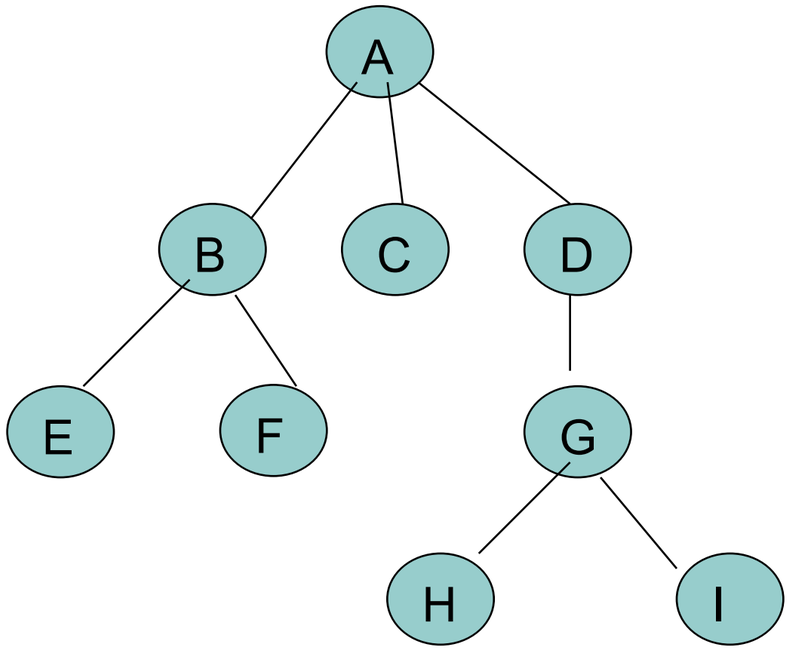
- Multiway Trees
- Disjoint-set

Multiway Trees: Definitions

- A (free) tree is any set of points (called vertices) and any set of pairs of distinct vertices (called edges or branches) such that (1) there is a sequence of edges (a path) from any vertex to any other, and (2) there are no circuits, that is, no paths starting from a vertex and returning to the same vertex.
- A rooted tree is a tree in which one vertex, called the root, is distinguished.
- An ordered tree is a rooted tree in which the children of each vertex are assigned an order.
- A forest is a set of trees. We usually assume that all trees in a forest are rooted.
- An orchard (also called an ordered forest) is an ordered set of ordered trees.

Tree Representations

- Parent Method (双亲表示法)



index	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	3
7	H	6
8	I	6

Easy to get a node's parent, but difficult to get its children

Tree Representations

```
/* 树的双亲表示法结点结构定义 */
#define MAX_TREE_SIZE 100
typedef char TElemType; /* 树结点的数据类型 */

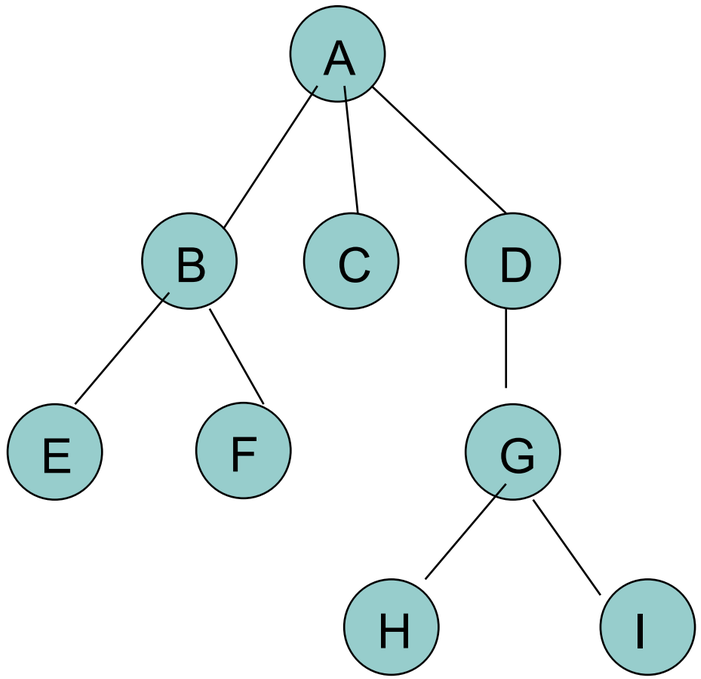
struct PTNode /* 结点结构 */
{
  TElemType data;    /* 结点数据 */
  int parent;        /* 双亲位置 */
};

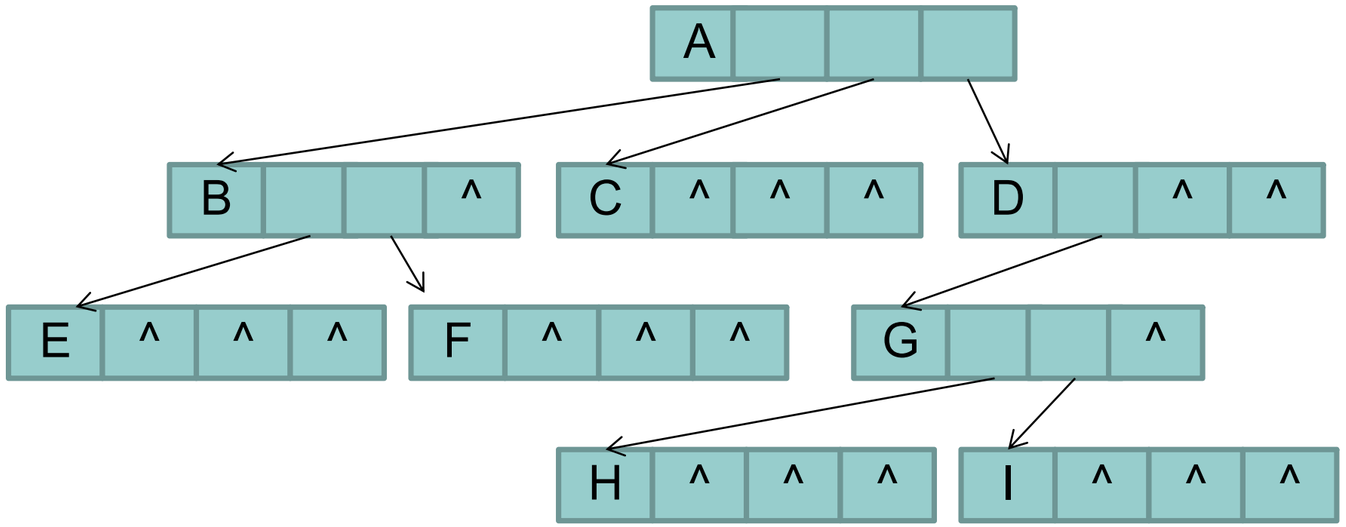
Struct PTree         /* 树结构 */
{
  PTNode nodes[MAX_TREE_SIZE]; /* 结点数组 */
  int r, n;          /* 根节点的位置和结点数 */
};
```

Tree Representations

- Multiple links（多重链表表示法）
  - Representation: The pointer field contains d pointers, where d is the maximum degree of every nodes.

data	child 1	child 2	child 3	……	child d

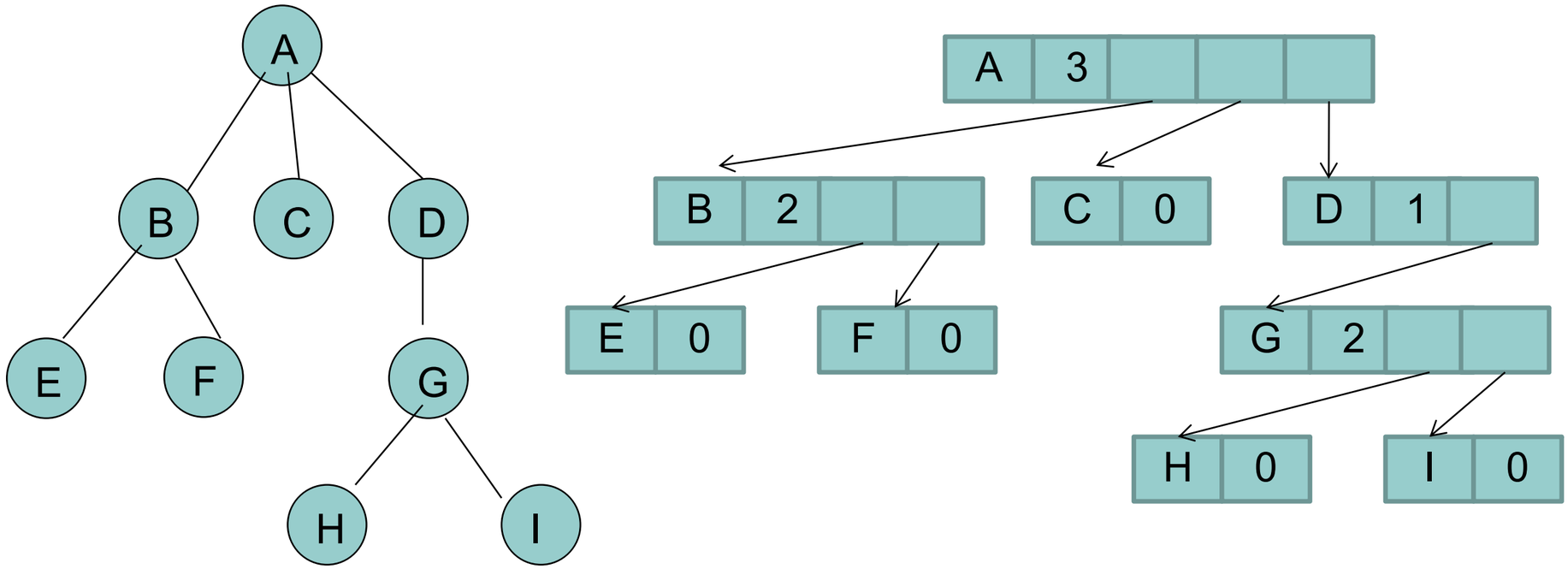




Tree Representations

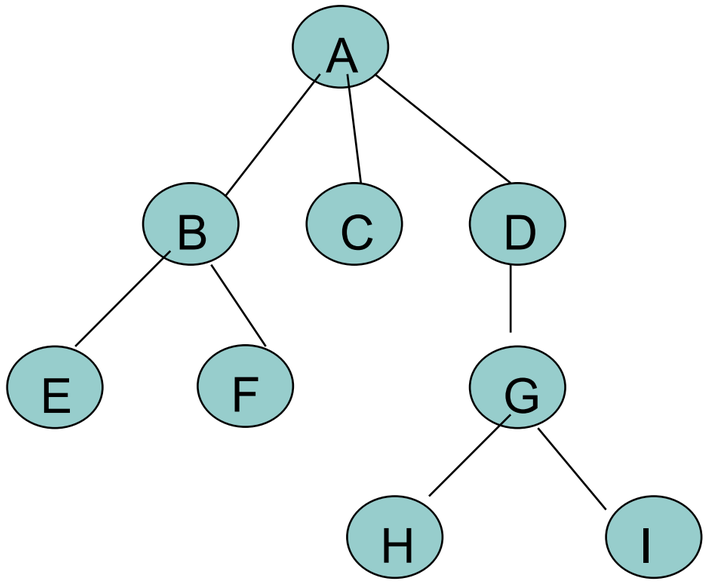
- Multiple links
  - Alternative representation: A number d indicates the degree of the node. And the pointer field contains d pointers.

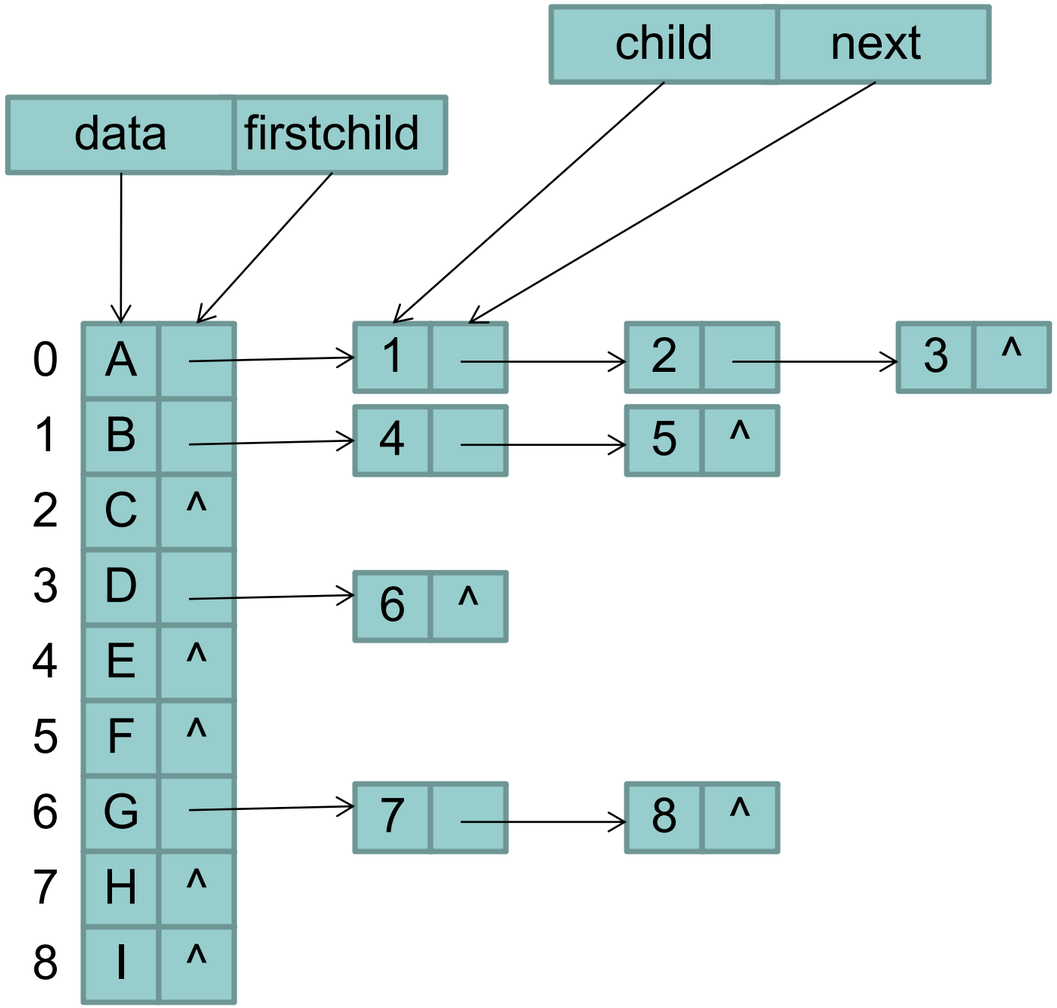
data	d	child 1	child 2	child 3	……	child d



Tree Representations

- Child-link（孩子链表表示法）





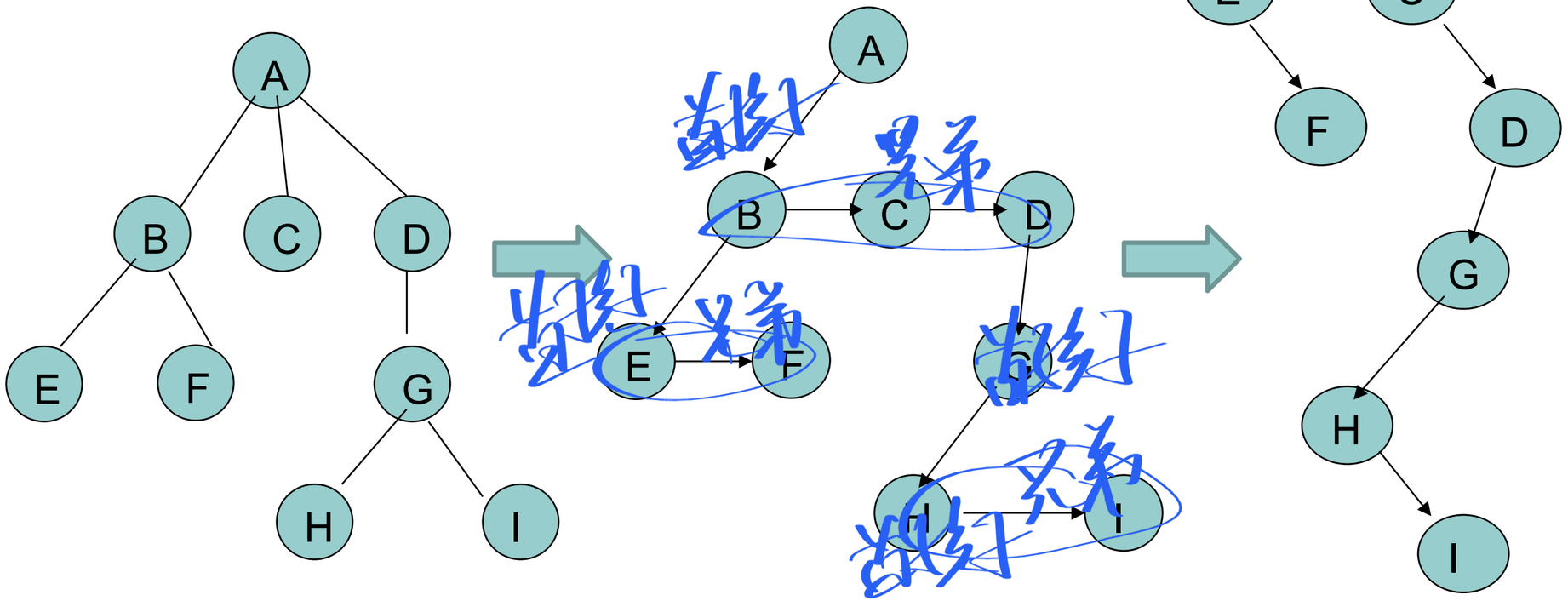
Tree Representations

```
#define MAX_TREE_SIZE 100
typedef char TElemType; /* 树结点的数据类型 */
typedef struct CTNode   /* 孩子结点结构 */
{
   int child;
   CTNode *next;
} *ChildPtr;
struct CTBox            /* 表头结构 */
{
   TElemType data;
   ChildPtr firstchild;
};
struct CTree            /* 树结构 */
{
   CTBox nodes[MAX_TREE_SIZE]; /* 结点数组 */
   int r, n; /* 根节点的位置和结点数 */
};
```

Tree Representations

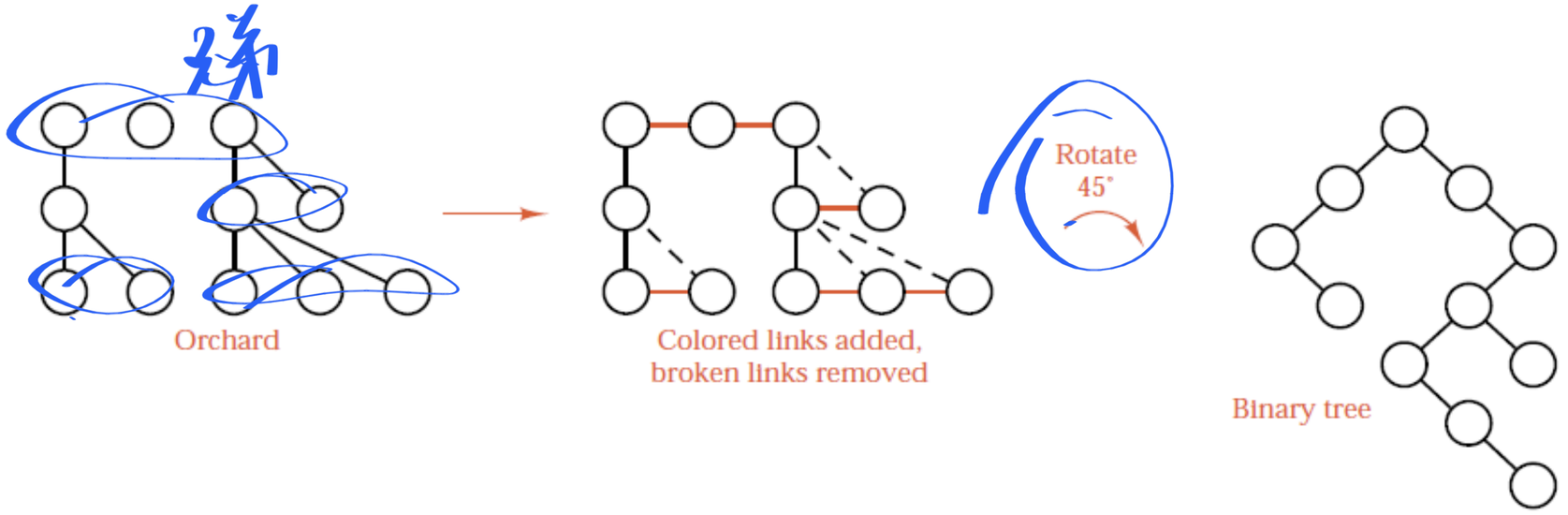
- First child next sibling（孩子兄弟表示法）

```
/* Node declarations for trees */
struct TreeNode
{
     TElemType data;
     TreeNode *firstChild;
     TreeNode *nextSibling;
};
```

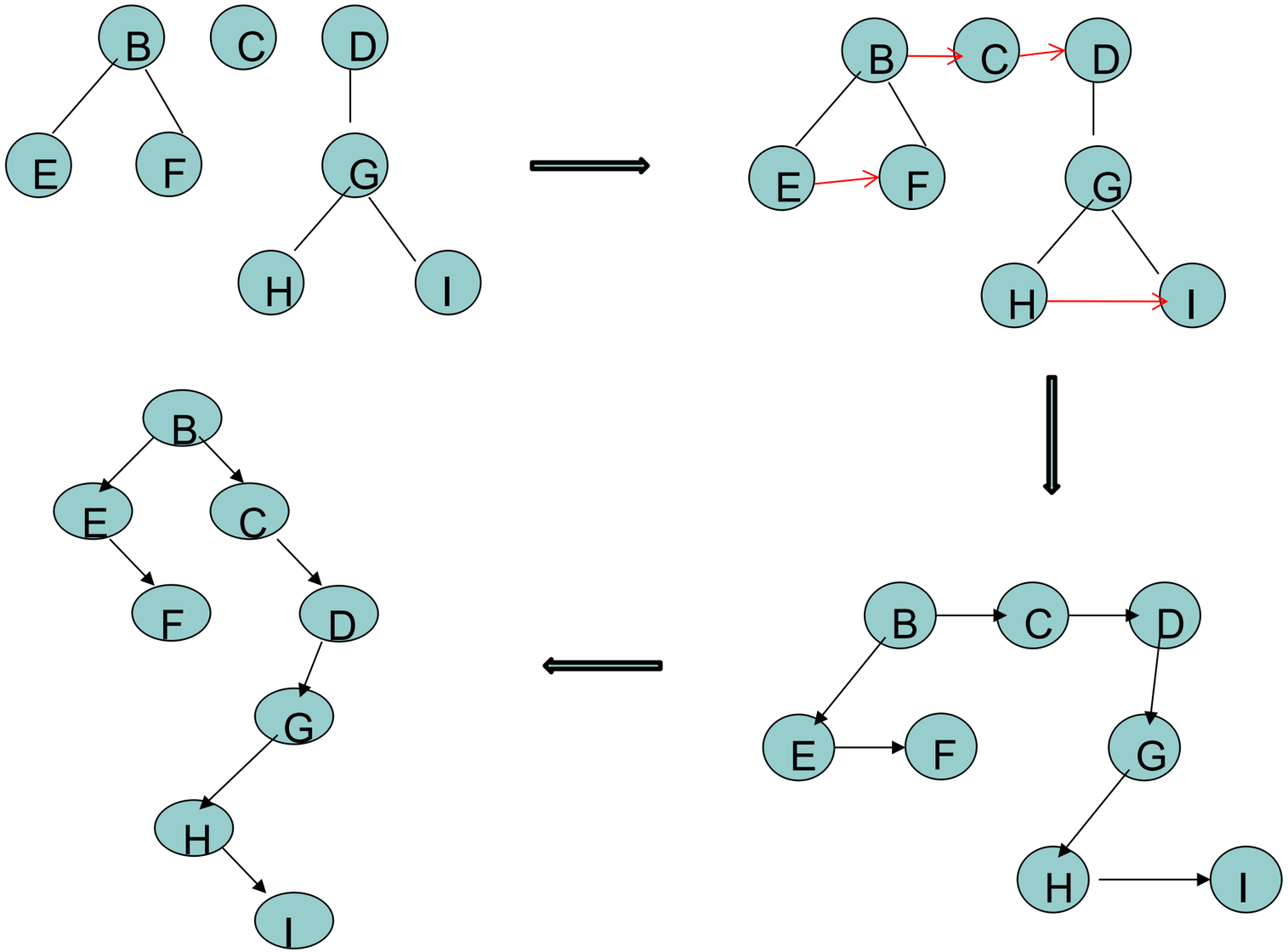


First-child-next-sibling Representation of Orchard

- Draw the orchard so that the first child of each vertex is immediately below the vertex.
- Draw a vertical link from each vertex to its first child, and draw a horizontal link from each vertex to its next sibling.
- Remove the remaining original links.
- Rotate the diagram 45 degrees clockwise, so that the vertical links appear as left links and the horizontal links as right links.

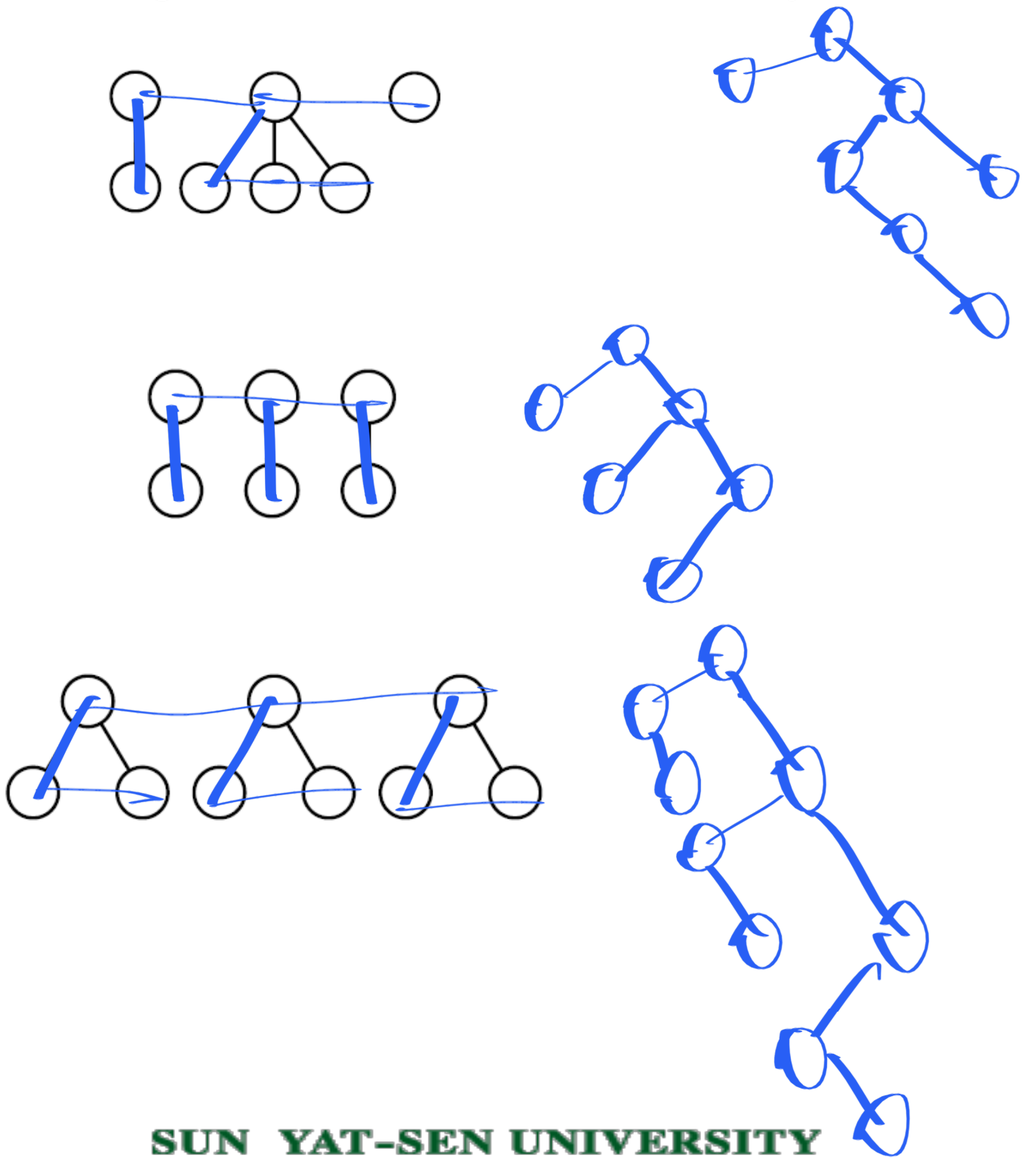


First-child-next-sibling Representation of Orchard



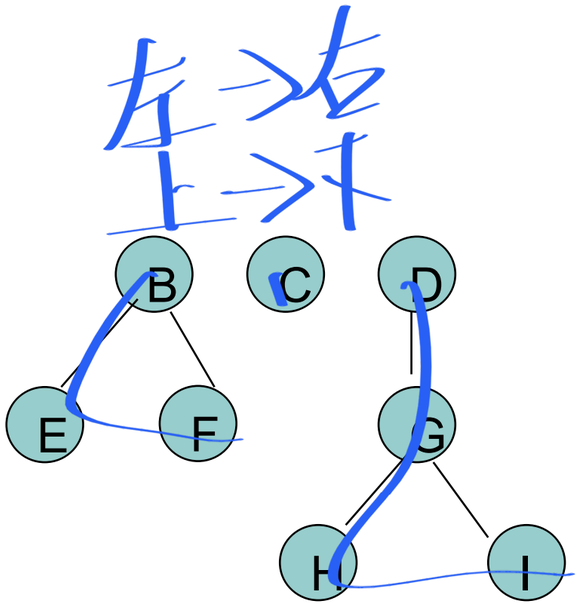
Exercise

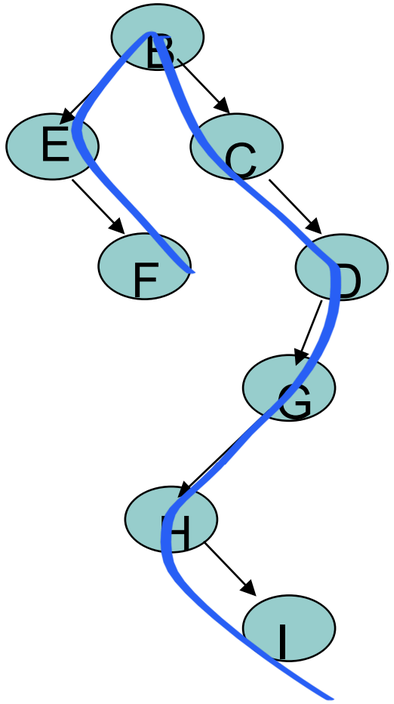
- Convert each of the following orchards into a binary tree.



Forest Traverse

- Preorder traverse: First root then children





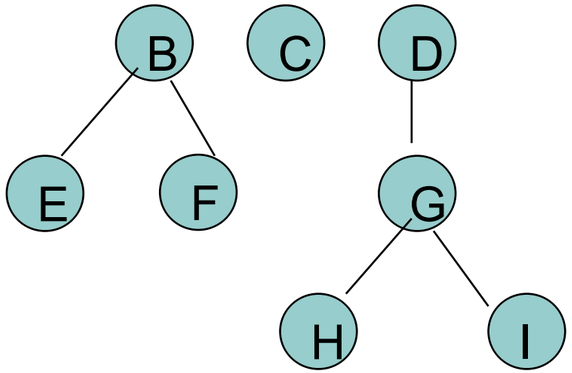
Preorder: BEFCDGHI

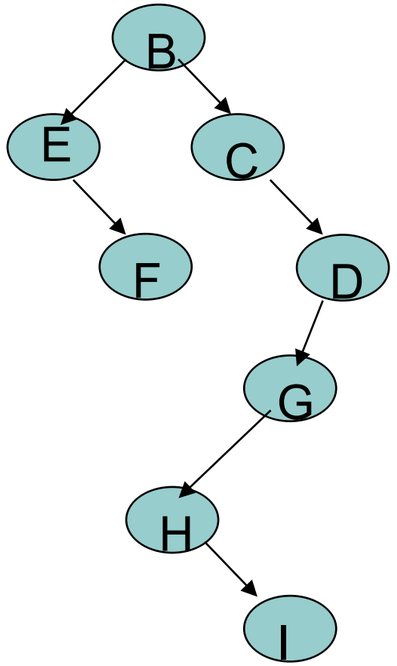
Preorder: BEFCDGHI

- Preorder traversal of the orchard is equivalent to the preorder traversal of the corresponding binary tree

Forest Traverse

- Postorder traverse: First children then root





Postorder: EFBCHIGD

Inorder: EFBCHIGD

- Postorder traversal of the orchard is equivalent to the inorder traversal of the corresponding binary tree

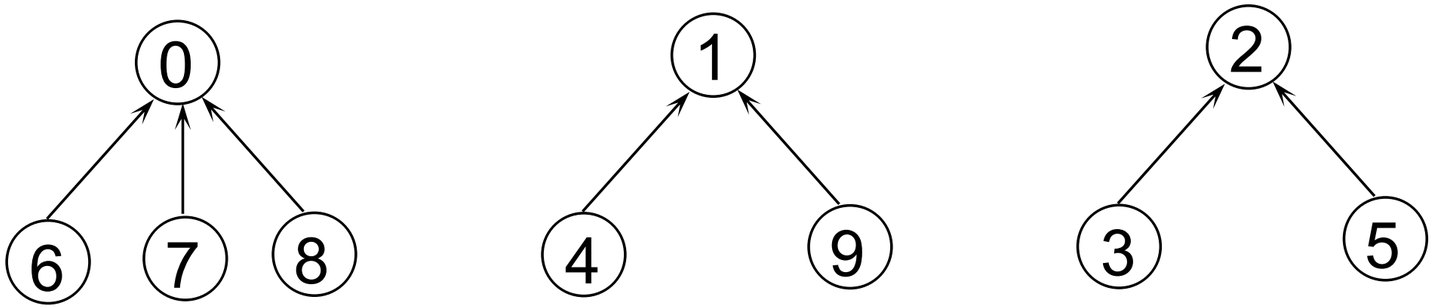
How to modify tree traversal into forest traversal?

```
template <class Entry>
void Binary_tree<Entry> :: recursive_preorder(Binary_node<Entry> *sub_root,
                                              void (*visit)(Entry &))
/* Pre:   sub_root is either NULL or points to a subtree of the Binary_tree.
   Post:  The subtree has been been traversed in preorder sequence.
   Uses:  The function recursive_preorder recursively */
{
   if (sub_root != NULL) {
      (*visit)(sub_root->data);
      recursive_preorder(sub_root->left, visit);
      recursive_preorder(sub_root->right, visit);
   }
}
```

```
template <class Entry>
void Binary_tree<Entry> :: recursive_postorder(Binary_node<Entry> *sub_root,
                                               void (*visit)(Entry &))
/* Pre:   sub_root is either NULL or points to a subtree of the Binary_tree.
   Post:  The subtree has been been traversed in postorder sequence.
   Uses:  The function recursive_postorder recursively */
{
   if (sub_root != NULL) {
      recursive_postorder(sub_root->left, visit);
      recursive_postorder(sub_root->right, visit);
      (*visit)(sub_root->data);
   }
}
```

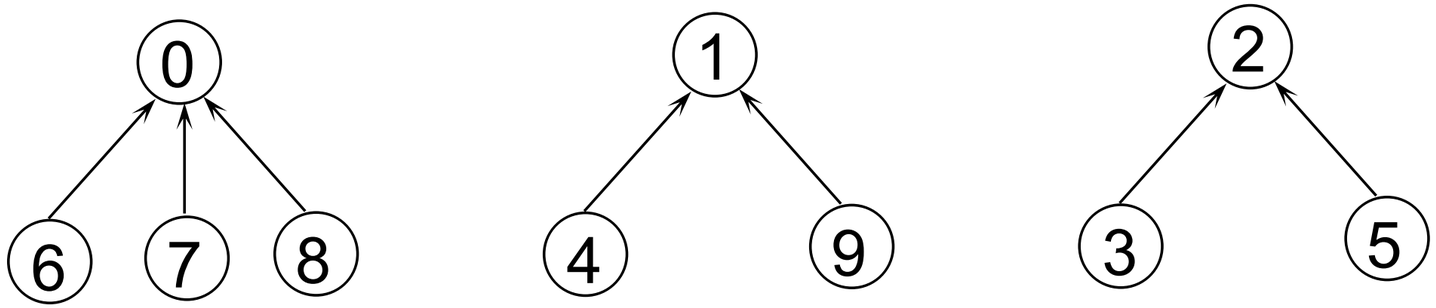
Disjoint-set(并查集) Forests

- Disjoint-set forests are data structures where each set is represented by a tree data structure, in which each node holds a reference to its parent node
- In a disjoint-set forest, the representative of each set is the root of that set's tree. *Find* follows parent nodes until it reaches the root. *Union* combines two trees into one by attaching the root of one to the root of the other.
- Example: S1 = {0, 6, 7, 8}, S2 = {1, 4, 9}, S3 = {2, 3, 5}



Disjoint-set Forests

- Use Parent method to represent disjoint-set forests.



- Two operations considered here
  - *Disjoint set union*

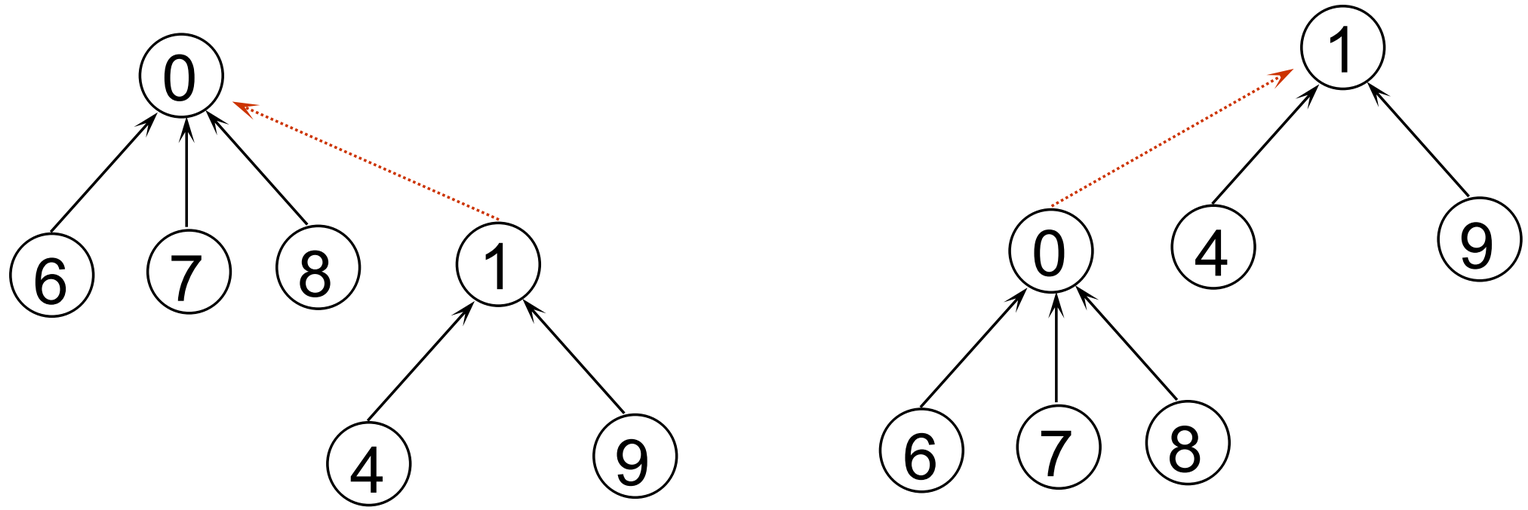
$$S_1 \cup S_2 = \{0,6,7,8,1,4,9\}$$

  - *Find*(*i*): Find the set containing the element *I,*

$$3 \in S_3,\ 8 \in S_1$$

Disjoint-set: Union

- Idea: Make one of trees a subtree of the other

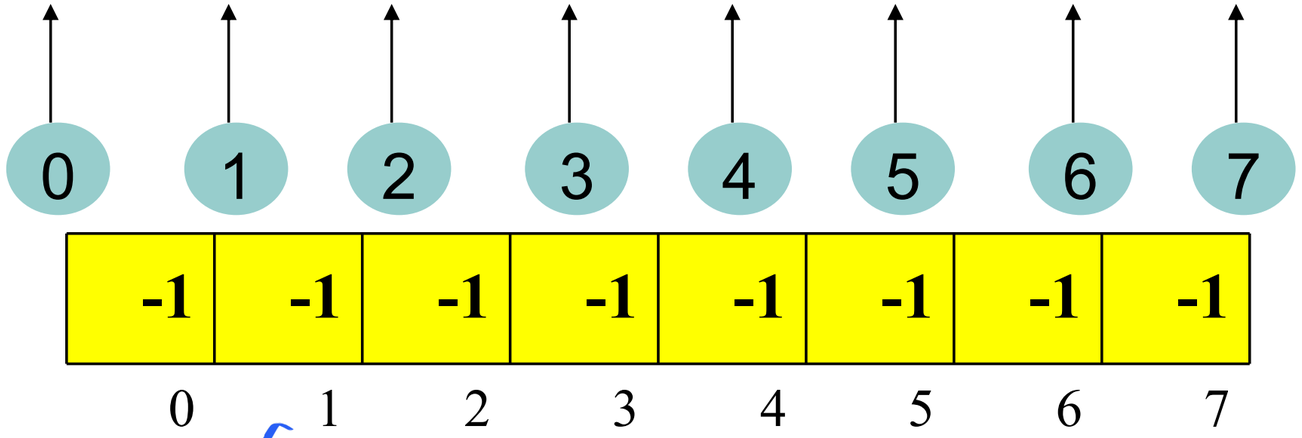


Possible representation for S_1 *union* S_2

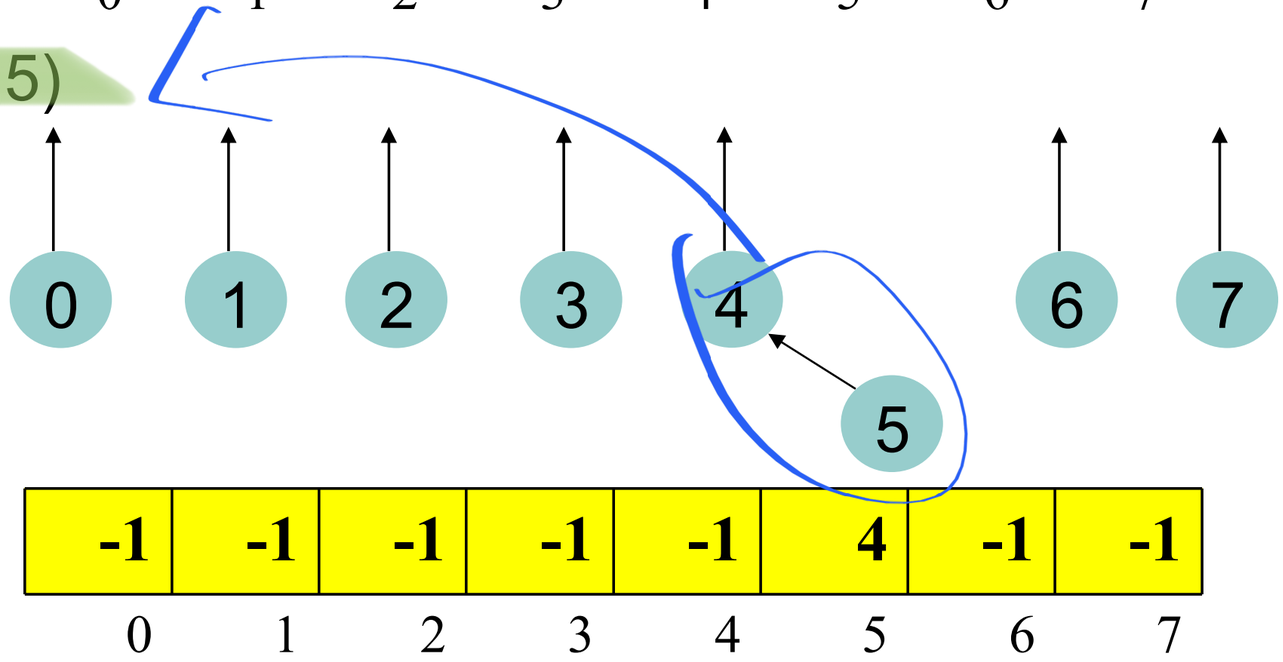
- The rule for merging two sets *A* and *B* is that if *A* is larger than *B* then merge the elements of *B* into *A* and update the elements that used to belong to *B*, and vice versa.
- In this case, *Find(i)* takes *O(log n)* time.

Disjoint-set Operations: Union

- Eight elements, initially in different sets

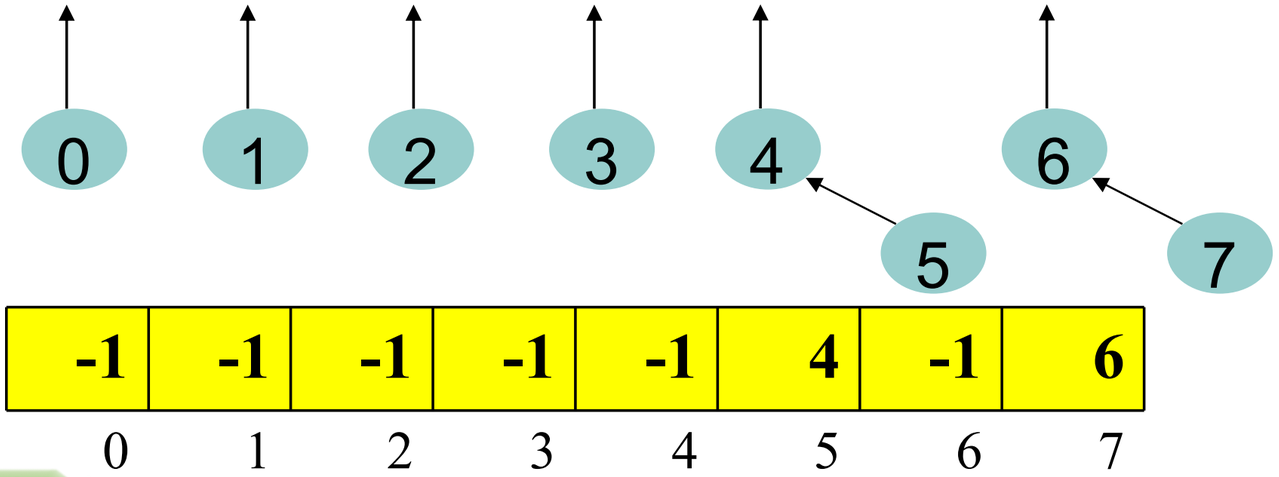


- After Union(4,5)

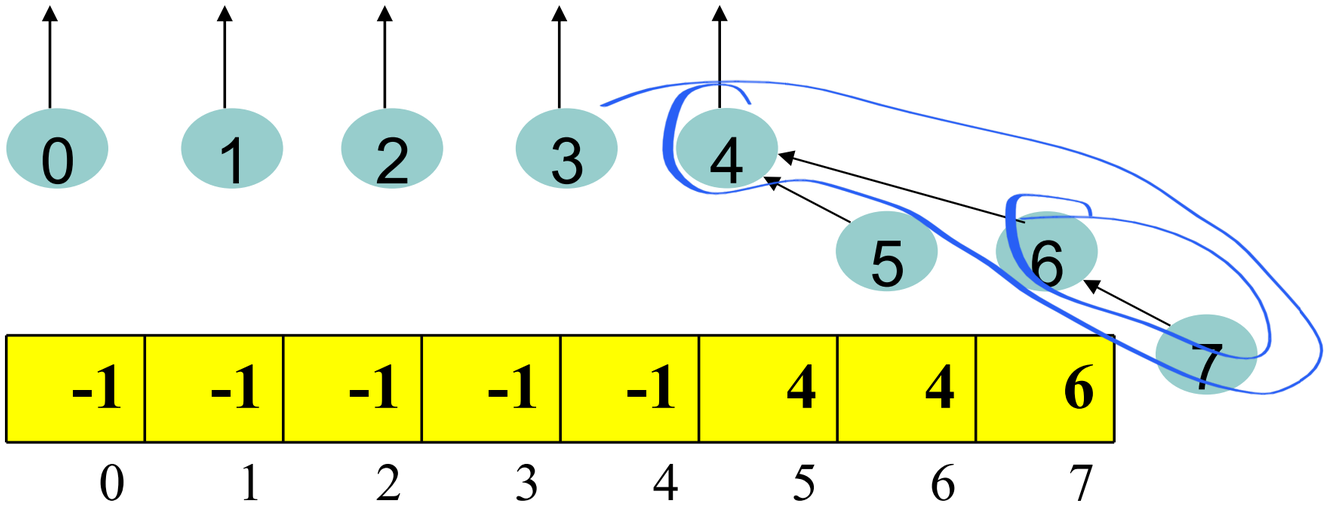


Disjoint-set Operations: Union

- After Union(6,7)

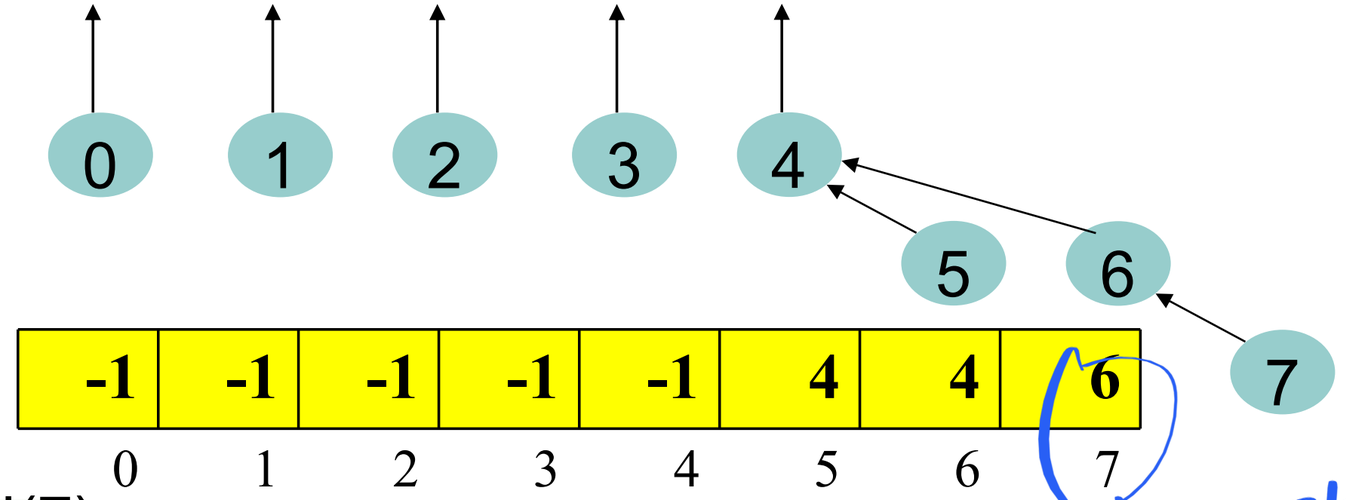


- After Union(4,6)

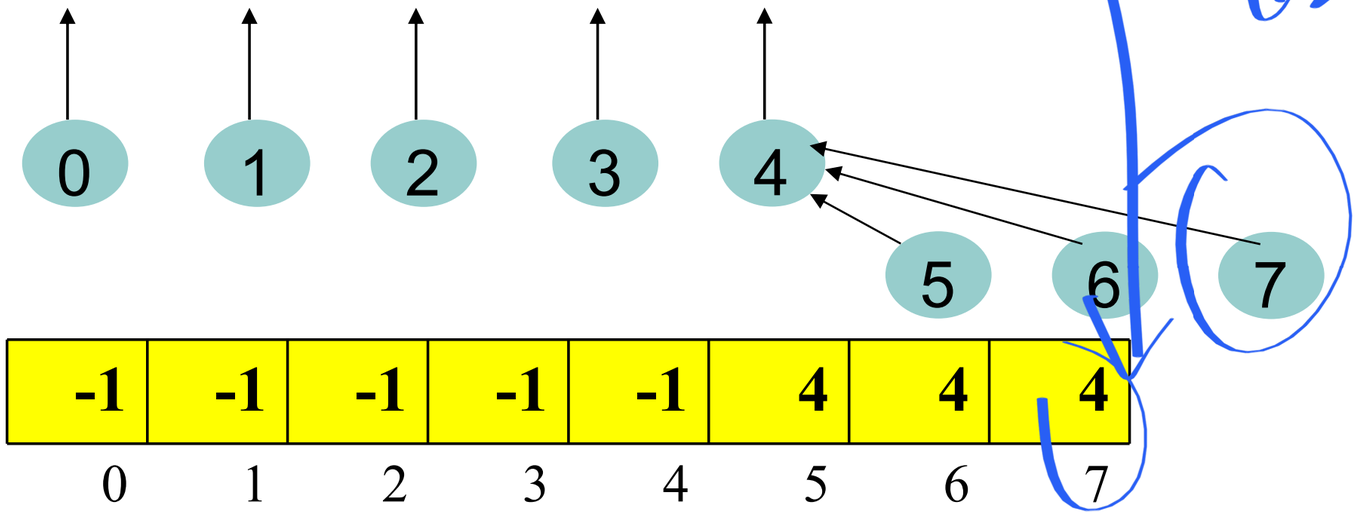


Disjoint-set Operations: Quick-Find

- Current

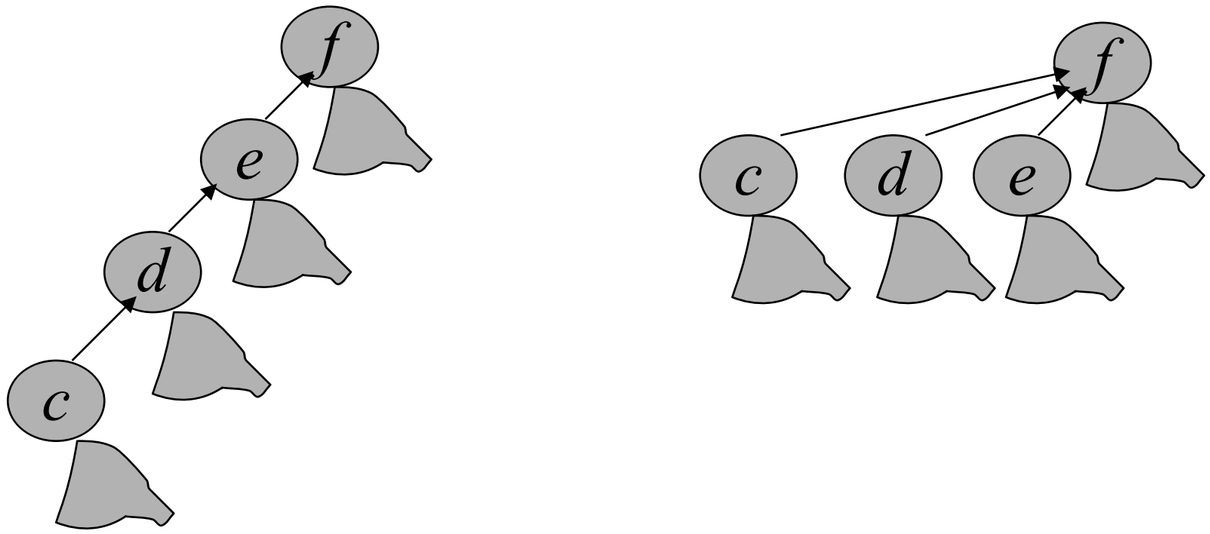


- After Find(7)



Union by Rank & Path Compression

- Union by Rank: Each node is associated with a rank, which is the upper bound on the height of the node (i.e., the height of subtree rooted at the node), then when UNION, let the root with smaller rank point to the root with larger rank.
- Path Compression: used in FIND-SET(*x*) operation, make each node in the path from *x* to the root directly point to the root. Thus reduce the tree height.



Disjoint-set Implementation

```
int p[maxn];

int find(int x) {
   return (p[x] == -1) ? x : p[x] = find(p[x]);
}

int union(int x, int y) {
   p[find(y)] = find(x);
}

int main() {
   for (int i = 0; i < n; i++)
        p[i] = -1;
   …
   union(x, y);
   if (find(x) == find(y)) …
}
```

Time Complexity

- MAKE-SETs requires O(V).
- FIND-SETs and UNIONs perform O(E) time. Assuming the implementation of disjoint-set data structure that uses union by rank and path compression, so amortized in O(E α(V)). (α为反ackerman函数)
- O(E α(V)) is almost linear.

Thank you!

